

Fine-Tuning Language Models With Just Forward Passes

Princeton's MeZO adapts billion-parameter models at the memory cost of inference — NeurIPS 2023

Fine-tuning is what turns a general pre-trained language model into a useful one, but it has a quiet, growing cost: memory. The reason is backpropagation. To compute a gradient, the standard training loop caches every intermediate activation on the forward pass, then stores a full gradient and a set of optimizer states on the backward pass. Together these can demand up to 12× the memory that plain inference needs on the same model. For a small model nobody notices. As models scale into the tens of billions of parameters, that factor becomes the wall you hit first.

The gap is easiest to feel in hardware terms. On a single 80GB A100 GPU, you can comfortably run inference on a 30B-parameter model. Try to fine-tune on that same card with Adam, and you are limited to roughly a 2.7B model, an order of magnitude smaller. So the frustrating situation is that the hardware can clearly hold and run the big model, it just cannot afford the bookkeeping that gradient-based training layers on top.

This paper, from Princeton University, asks whether that bookkeeping is truly necessary. Its answer is MeZO, a memory-efficient zeroth-order optimizer that fine-tunes a language model using only forward passes, with no backpropagation at all. The headline is that training then costs about the same memory as inference, while still matching gradient-based fine-tuning on many tasks. Figure 1 sets the stakes: across eleven tasks on OPT-13B, MeZO tracks full Adam fine-tuning closely while using only 1/12 of the memory.

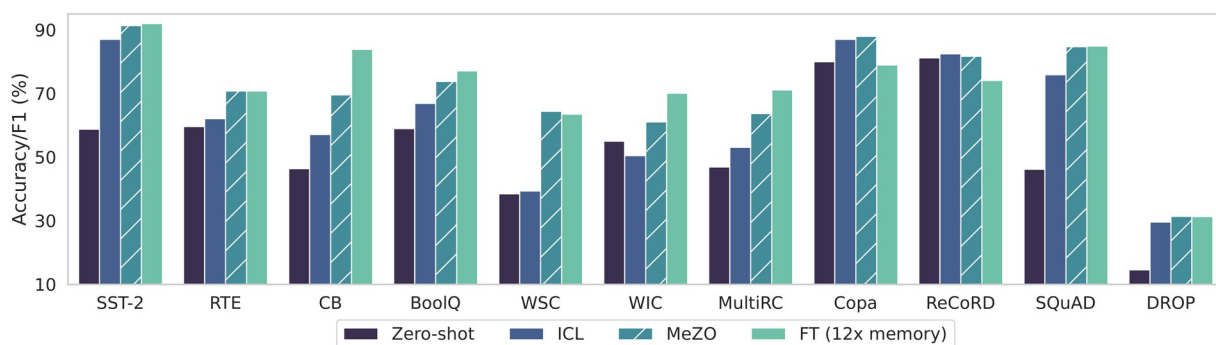


Figure 1. OPT-13B across eleven tasks: zero-shot, in-context learning (ICL), MeZO, and full fine-tuning with Adam (FT). MeZO clearly beats zero-shot and ICL and comes within about 1% of fine-tuning on 7 of 11 tasks, despite using only 1/12 of the memory.

Paper: <https://arxiv.org/abs/2305.17333> | Code: <https://github.com/princeton-nlp/MeZO>

Why fine-tuning got so memory-hungry

It helps to separate two costs. Inference only needs the model weights and a modest activation buffer for the tokens currently in flight. Backpropagation adds three things on top: the cached activations for the whole forward pass, a gradient of the same size as the parameters, and optimizer state such as Adam's running averages, which for Adam is another two copies of the parameters. Add it up and a training step can need many times the footprint of a forward pass. Parameter-efficient methods like LoRA and prefix tuning shrink the optimizer and gradient terms, but they do not remove the activation caching, and they still run backpropagation.

There is an old idea that sidesteps backpropagation entirely: zeroth-order optimization, which estimates a gradient purely from function values. You perturb the parameters, look at how the loss changes, and infer a descent direction from those two numbers. The catch, and the reason the field mostly ignored it for large models, is a pessimistic piece of classical theory: the convergence rate of zeroth-order methods was expected to degrade in proportion to the number of parameters. For a model with billions of them, that sounds hopeless. The paper's central empirical claim is that this pessimism does not hold when you are fine-tuning a strong pre-trained model on a downstream task.

MeZO: a gradient estimate that fits in an inference footprint

MeZO is built on a classical estimator called SPSA (simultaneous perturbation stochastic approximation). Each step samples a single random direction z with the same shape as the parameters, drawn from a standard Gaussian. It adds a small step $\epsilon \cdot z$ to every parameter and records the loss, then subtracts to get the loss on the other side. The difference between those two losses, divided by 2ϵ , is a single scalar, the projected gradient, and the parameter update simply moves along z scaled by that scalar and the learning rate.

Concretely, the SPSA gradient estimate is $\nabla L(\theta) = [L(\theta + \epsilon z) - L(\theta - \epsilon z)] / (2\epsilon) \cdot z$ with z drawn from $N(0, I)$, and the parameter update is the ordinary SGD step $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$ applied along that single sampled direction.

Written naively, this still doubles memory, because you have to store the full perturbation vector z to add it, evaluate, and subtract it again. The engineering insight that makes MeZO practical is to never store z . Instead it stores only the random seed, and regenerates z on the fly each time it is needed, once to perturb positively, once to perturb negatively, and once to apply the update. Because the same seed reproduces the same z , MeZO perturbs and resets the weights in place, so the entire perturb-evaluate-update cycle runs inside the memory of a plain forward pass. Two forward passes per step, no gradients, no activation cache, no optimizer state.

The memory story, measured

Figure 3 makes the payoff concrete. It profiles GPU memory on MultiRC across OPT sizes from 1.3B to 30B, comparing zero-shot inference, in-context learning, full fine-tuning, prefix-tuning fine-tuning, and MeZO. MeZO's bar sits right on top of the inference bar at every scale, while the fine-tuning bars climb far above it. The saving grows with model size, reaching up to

12× at 13B, and it is what lets a single 80GB A100 fine-tune a 30B model where backpropagation would fit only 2.7B.

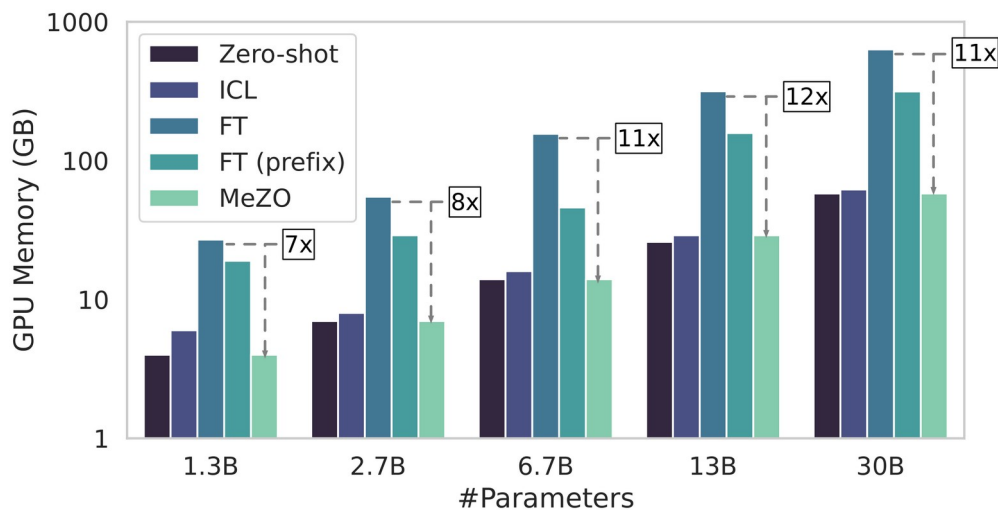


Figure 3. GPU memory on MultiRC (about 400 tokens per example) for different OPT sizes and tuning methods. MeZO uses the same memory as inference and delivers up to 12× savings over standard fine-tuning.

Table 1. Memory saving of MeZO over full fine-tuning (Adam), by OPT model size, read from Figure 3. MeZO's own footprint equals inference at every scale.

OPT model	Saving vs. fine-tuning
1.3B	7×
2.7B	8×
6.7B	11×
13B	12×
30B	11×

Does it actually match fine-tuning?

Memory savings only matter if quality holds up, so the evaluation is deliberately broad. On the autoregressive side it uses OPT models from 1.3B up to 66B on SuperGLUE classification, multiple-choice tasks such as COPA and ReCoRD, and generation tasks such as SQuAD and DROP. On the masked-language-model side it uses RoBERTa-large (350M parameters) on six sentence-classification and inference tasks, in both a few-shot regime ($k=16$) and a many-shot regime ($k=512$). The baselines are the honest ones: zero-shot, in-context learning, linear probing, and full Adam fine-tuning.

The OPT-13B result in Figure 1 is the clearest single summary. MeZO comfortably outperforms zero-shot prediction, in-context learning, and linear probing, and it lands within about 1% of full fine-tuning on 7 of 11 tasks, at 1/12 of the memory. On RoBERTa-large the same pattern holds at smaller scale: MeZO stays close to fine-tuning, coming within roughly 5% in the many-shot $k=512$ setting. And the memory win does not cost wall-clock time in practice; in the authors' implementation MeZO needs about 2× fewer GPU-hours than Adam fine-tuning for a 30B model.

It composes with the tricks you already use

MeZO is not an alternative to parameter-efficient tuning, it stacks on top of it. Figure 2 reports RoBERTa-large with MeZO applied to full parameters, to LoRA, and to prefix-tuning, alongside their fine-tuning counterparts. The MeZO variants sit close to each other and to fine-tuning, which means you can combine forward-pass-only training with LoRA or prefix-tuning and keep the accuracy. Because MeZO only ever reads the loss value and never a true gradient, it can also optimize objectives that have no gradient at all, such as accuracy or F1 directly, and against the prior zeroth-order method BBTv2 it improves accuracy by up to 11% absolute.

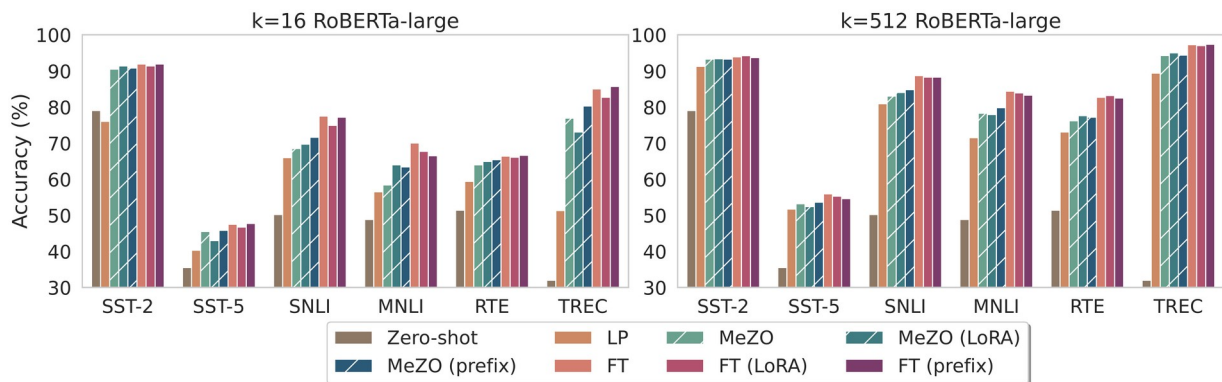


Figure 2. RoBERTa-large: zero-shot, linear probing (LP), and MeZO and fine-tuning (FT) with full parameters, LoRA, and prefix-tuning. MeZO beats zero-shot and LP and approaches fine-tuning (within about 5% at k=512) with far less memory.

Why the pessimism was wrong

The paper also explains why a method that classical theory said should crawl instead converges at a usable rate. The short version is that when you start from a well pre-trained model, the loss landscape around the current weights is effectively low-dimensional. The rate of MeZO is governed by that effective local structure, the local rank of the landscape, rather than by the raw parameter count. This is why the parameter-count pessimism does not bite: fine-tuning does not have to search the full billion-dimensional space, only the small subspace that matters near a good solution.

The takeaway

The lasting message is simple: fine-tuning no longer strictly requires backpropagation. With MeZO you can adapt very large language models using only forward passes, at the memory cost of inference, and still match gradient-based quality on many tasks, which in turn lets you train models an order of magnitude larger on the same hardware. The honest boundary is worth stating too. MeZO closes most of the gap to fine-tuning but not all of it, it can need more steps to get there, and the strong convergence rests on starting from a capable pre-trained model. Within those limits, it turns the memory budget of inference into a training budget, and that is a genuinely useful trade.